Informe técnico: Focos de calor en Colombia (NASA FIRMS)

7 de octubre de 2026 · Juan Pablo Bernal y Samuel Muñoz

Resumen y arquitectura

El sistema descarga detecciones satelitales de focos de calor en Colombia (2012–2026), las limpia, las guarda como GeoJSON y las sirve por una API con consultas geoespaciales, todo en contenedores que Jenkins redespliega solo.

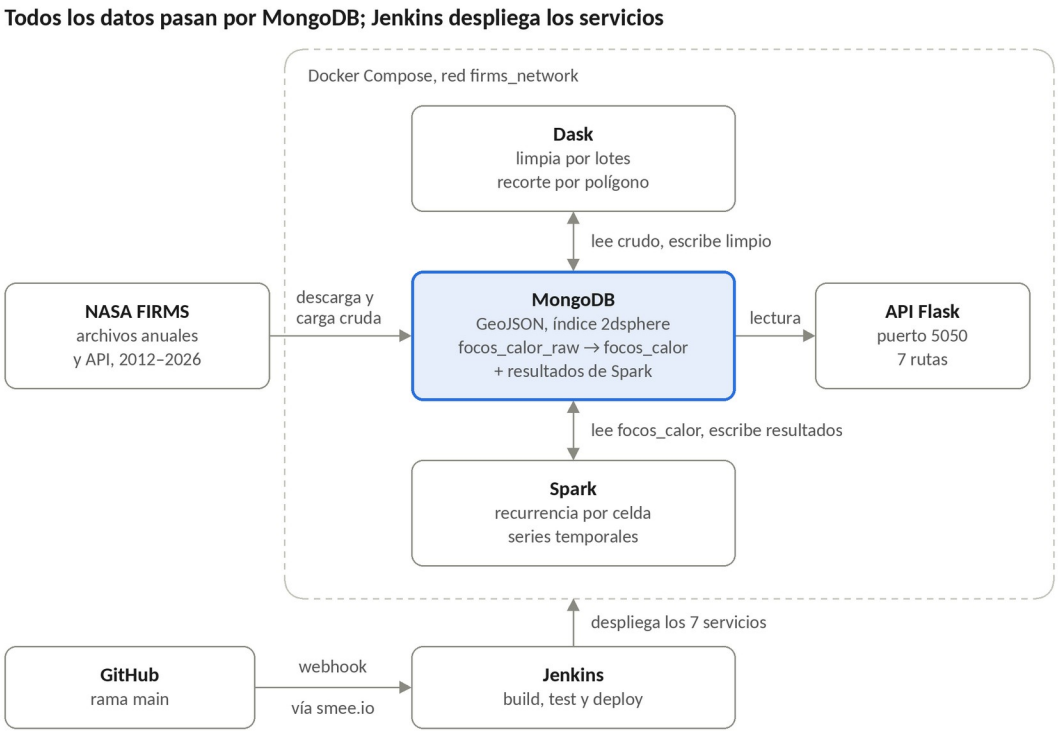


Figura 1. Arquitectura del sistema: MongoDB en el centro; Jenkins despliega los servicios.

MongoDB está en el centro: Dask lee lo crudo y escribe `focos_calor`; Spark lee `focos_calor` y escribe tres colecciones de resultados; la API Flask lee `focos_calor` y esas colecciones. Jenkins construye, prueba y despliega los siete contenedores de Docker Compose cada vez que cambia `main`. Es el proyecto del curso de Big Data de la IUE, de Juan Pablo Bernal y Samuel Muñoz.

Datos y limpieza

De 2.549.897 detecciones descargadas se conservaron 2.333.350 dentro de Colombia: se descartaron 216.547 (8,49 %) por caer fuera del territorio y ninguna por coordenadas o fecha inválidas. La fuente es NASA FIRMS (sensores VIIRS), con datos hasta el 2 de octubre de 2026.

Origen	Periodo	Sensor	Detecciones crudas
Archivos anuales por país	2012–2024	Suomi-NPP (2012–2024) y NOAA-20 (2018–2024)	2.187.607
API, calidad científica (SP)	2025-01-01 a 2026-06-30	NOAA-20	336.620
API, tiempo casi real (NRT)	desde 2026-07-01	NOAA-20	25.670

Casi todo lo descartado viene de la API, que no recorta por país y entrega una caja de coordenadas. Se estima que unas 211.000 de sus 362.290 detecciones (≈ 58 %) cayeron fuera de Colombia, calculado por el orden de los lotes. En los archivos anuales, que NASA ya recorta, se perdió ≈ 0,25 % (unas 5.200 filas).

La limpieza usa Dask sobre lotes de 100.000 documentos: lee `focos_calor_raw` y escribe `focos_calor`. Estas son sus decisiones y el motivo de cada una:

Decisión	Qué se hizo	Por qué
Recorte por territorio	Se descartan las detecciones fuera del polígono de Colombia (geoBoundaries, 733 vértices)	La API devuelve una caja con partes de países vecinos. El polígono simplificado no incluye San Andrés ni Providencia: sus 9 detecciones crudas se descartan
Validación	Se validan coordenadas y fecha	Ninguna de las 2.549.897 filas falló la validación
Hora	`fecha_hora_utc` (la hora real) y `fecha_hora_colombia` (UTC−5)	La local se guarda como fecha sin zona, que MongoDB muestra como si fuera UTC; Spark se fija en UTC para no restar 5 horas otra vez
Esquema entre fuentes	`version` se guarda como número y `version_original` conserva «2.0NRT»; `type` queda nulo en NRT	Las detecciones NRT no traen `type` y `version` cambia de tipo entre fuentes
Geometría	`location` es un punto GeoJSON con orden [longitud, latitud]	Con el orden invertido las consultas devuelven vacío sin dar error
Confianza baja	No se elimina: se conserva `confidence` (`l`, `n`, `h`)	Se puede filtrar al consultar; borrarla no tendría vuelta atrás
Duplicados	No se eliminan	Píxeles vecinos del mismo fuego son detecciones distintas, y no hay duplicados exactos ni solape entre SP y NRT
Sensor en la API	Solo NOAA-20	Suomi-NPP y NOAA-20 ven casi lo mismo desde 2019; pedir ambos duplicaría
Campo `type`	No se usa para separar incendios de fuentes fijas	Es poco fiable: junto a la mina del Cerrejón hay detecciones con `type` 2 (fuente estática) y otras, a metros, con `type` 0 (vegetación). Se usa la recurrencia por celda

Modelado y consultas geoespaciales

Las 2.333.350 detecciones están en MongoDB como GeoJSON con índice `2dsphere`, y las tres consultas geoespaciales pedidas devuelven resultados coherentes entre sí.

Cada documento guarda `location` como un punto GeoJSON con `coordinates: [longitud, latitud]`. Además del `2dsphere` hay índices sobre `fecha_hora_colombia`, `acq_date`, `confidence` y `satellite`.

Endpoint	Operador	Parámetros
`GET /focos/cerca`	`$near`	`lat`, `lon`, `radio_km` (0,1 a 100), `limite` (1 a 1000), `confianza`, `frp_min`
`POST /focos/poligono`	`$geoWithin`	cuerpo JSON con `poligono` (GeoJSON Polygon), `limite`, `confianza`
`GET /agregados/cerca-mensual`	agregación `$geoNear`	`lat`, `lon`, `radio_km` (0,1 a 50)

Ningún valor está fijo en el código: los parámetros inválidos devuelven 400 con un mensaje, y un fallo de MongoDB devuelve 503. Como `$near` no sirve para contar documentos, el total de `/focos/cerca` se calcula aparte con `$geoWithin`.

La coherencia se comprobó en la zona minera del Cerrejón (latitud 11,13; longitud −72,58), con radio de 5 km: `/focos/cerca` cuenta 19.900 detecciones y la suma de los 176 meses de `/agregados/cerca-mensual` también da 19.900. Con `frp_min=10` el total baja a 328. Esa igualdad entre dos consultas distintas es una de las pruebas automáticas.

Tiempo de respuesta. Las cuatro consultas medidas respondieron en menos de 0,5 s en tres corridas seguidas, entre 11 ms y 476 ms.

Consulta	Corrida 1	Corrida 2	Corrida 3
`GET /focos/cerca` (5 km, 100 resultados)	476 ms	109 ms	115 ms
`GET /agregados/cerca-mensual` (5 km)	183 ms	198 ms	271 ms
`POST /focos/poligono` (cuadrado de unos 11 km, 100 resultados)	182 ms	151 ms	133 ms
`GET /spark/serie-mensual`	11 ms	12 ms	11 ms

Se midió con `curl` desde el dev container, dentro de la red de Docker, en un solo punto (el del Cerrejón) y sin carga concurrente. No se midieron otros puntos ni radios.

Procesamiento con Spark

Dos jobs de Spark leen `focos_calor` desde MongoDB con el conector oficial y escriben tres colecciones nuevas: 327.057 celdas con su recurrencia y dos series temporales. La API las sirve en tres rutas `/spark/*`.

Recurrencia por celda (`celdas_recurrencia`; 1 min 36 s en la primera corrida). El territorio se divide en celdas de 0,01° (≈ 1,1 km en latitud; en longitud varía) y por celda se cuentan las detecciones y los días, meses y años distintos. Una celda es candidata a fuente estática si suma al menos 500 detecciones, 300 días, 60 meses y 7 años: son 60 celdas, con 84.272 detecciones (3,61 %). La primera es la del Cerrejón, con 7.598 detecciones en 2.151 días y 14 años. Es una heurística para señalar candidatas, no una prueba.

Series temporales (`serie_mensual` y `serie_horaria`; 2 min 1 s). Se usa un solo sensor por año, Suomi-NPP hasta 2018 y NOAA-20 desde 2019: en 2020 ambos cuentan casi lo mismo (147.175 y 145.424) y sumarlos duplicaría esos años. Quedan 1.617.638 detecciones, 55.106 de ellas (3,41 %) en celdas candidatas, repartidas en 178 meses de 2012-01 a 2026-10.

- Los cinco meses con más detecciones son de enero a marzo: 2018-02 (58.821), 2020-03 (54.580), 2016-03 (41.157), 2022-01 (41.094) y 2020-02 (38.396).
- Octubre de 2026 es un mes parcial, con 76 detecciones hasta el 2 de octubre; no es una caída.
- Por hora local, el 83 % de las detecciones cae entre las 11 y las 14 horas y el 17 % entre las 0 y las 2. VIIRS es de órbita polar: eso refleja sus horarios de paso, no el ciclo diario de los incendios.

Comparativa Dask vs Spark

En este montaje, Dask terminó la agregación por celda entre 3,8 y 4,3 veces más rápido que Spark y usó unas 2,5 veces menos memoria, pero eso no prueba que un motor sea mejor: la lectura de datos es distinta en cada uno.

La operación es la agregación por celda de la sección anterior, sobre `focos_calor` completo. Cada corrida compara su resultado con el de referencia (327.057 celdas, 60 candidatas y 84.272 detecciones en ellas), y las 17 corridas coincidieron. Spark lee con el conector oficial; Dask, con un lector propio de 15 tareas (una por año, de 2012 a 2026) que filtra por fecha y trae solo 3 campos. Todo corre en una máquina, con Docker en 11,68 GiB y 12 CPUs.

Tiempo (segundos). Cada configuración se corrió tres veces seguidas. La primera fue la más lenta en las cuatro; sospechamos el calentamiento de MongoDB, sin comprobarlo.

Configuración	Corrida 1	Corrida 2	Corrida 3	Promedio de las 3	Promedio de la 2 y 3
Spark, 1 core	97,85	46,43	47,99	64,09	47,21
Spark, 2 cores	63,77	44,41	56,15	54,78	50,28
Dask, 1 worker	14,96	11,99	9,80	12,25	10,90
Dask, 2 workers	37,06	15,71	10,96	21,24	13,34

La razón de 3,8 a 4,3 sale del promedio de la 2 y 3, comparando 1 con 1 y 2 con 2. Con el promedio de las tres, Spark con 2 cores (54,78 s) queda por debajo de Spark con 1 core (64,09 s): la conclusión depende de cómo se promedie.

Memoria (MiB). Se muestreó `docker stats` mientras corría cada job, con los contenedores recién reiniciados.

Motor	Contenedor	Reposo	Pico	Subida	Muestras
Spark, 1 core	`firms_spark_worker`	213	1.332	1.119	105
Spark, 2 cores	`firms_spark_worker`	198	1.529	1.331	47
Dask, 1 worker	`firms_dask_worker_1`	147	570	424	25
Dask, 2 workers	`firms_dask_worker_1`	151	408	257	14
Dask, 2 workers	`firms_dask_worker_2`	151	436	285	14

La subida de Spark fue entre 2,5 y 2,6 veces la de Dask (1.331 contra 257 + 285 con dos unidades; 1.119 contra 424 con una).

Qué no sostienen las mediciones.

- Que un motor sea mejor. No sabemos cuánto de la diferencia es del motor y cuánto del lector: Spark lee todos los campos con el conector oficial; Dask, solo tres y con un filtro por fecha indexado.
- Que más recursos ayuden. Con 2 cores Spark no fue más rápido que con 1 (3 s de diferencia a favor del 1), ni Dask con 2 workers que con 1 (2,4 s a favor del 1). Son diferencias dentro de la variación entre corridas, y no se determinó la causa.
- Cifras exactas de memoria. Hay una medición por configuración, con 14 a 105 muestras de `docker stats` y sin contar el driver.

Cuándo conviene cada uno, según lo medido. Para una agregación que se puede partir por una clave natural (aquí, el año) y cuyos datos caben en memoria, Dask fue más rápido y más liviano, pero hubo que escribir el lector de MongoDB y repartir el trabajo a mano. Spark trajo el lector listo y expresó la operación como una consulta de DataFrame, a cambio de más memoria (un pico de 1,3 a 1,5 GB en el worker) y sin ventaja de tiempo a este tamaño. Qué pasa con datos que no caben en una máquina, o con varias máquinas, no se midió.

Integración y despliegue continuo

Cada cambio en `main` se construye, se prueba y se despliega solo: se comprobó con un cambio real (build #13), un fallo provocado (#15), un webhook de GitHub (#19, #20, #21 y #23) y su respaldo (#22).

El `Jenkinsfile` tiene cuatro etapas: construir las imágenes; levantar MongoDB y la API y correr 47 pruebas con `pytest` dentro de la red `firms_network`; desplegar con `docker compose up -d --build`; y listar los servicios. Si una prueba falla, la etapa de despliegue no se ejecuta. Las llaves (`MAP_KEY` y el token de GitHub) son credenciales de Jenkins; `.env` está fuera de Git y `.env.example` no lleva valores.

Jenkins se activa con un webhook de GitHub. Como corre en una máquina local, el evento pasa por un relé (smee.io) y un cliente en el dev container que lo reenvía; la consulta cada minuto queda como respaldo: con el relé apagado, el build #22 arrancó por ella. Sin un parámetro al final del destino, el cliente recortaba la barra final: Jenkins respondía con una redirección y el POST llegaba como GET (error 405).

Build	Qué se probó	Resultado
#13	Cambio real: filtro `frp_min` en `/focos/cerca`	Verde, 47 pruebas. Arrancó 24 s después del commit y duró 46 s; con `frp_min=10` la consulta pasó de 19.900 a 328 detecciones
#15	Fallo provocado: una prueba que falla a propósito	Rojo en Test, Deploy omitido, la API siguió con la misma imagen
#19	Webhook	Causa «Started by GitHub push»; arrancó como máximo 12 s después del push; duró 30 s
#20	Fusión de un pull request en `main`	Verde en 31 s, con dos causas: el webhook y la consulta cada minuto
#21	Webhook de nuevo	Verde, 47 pruebas; causa «Started by GitHub push» sola, como máximo 16 s después del push; duró 1 min 36 s
#22	Respaldo: relé apagado	Verde, 47 pruebas; causa solo «Started by an SCM change» (la consulta cada minuto), entre unos 13 y 72 s después del commit; duró 36 s
#23	Webhook con el relé reiniciado	Verde, 47 pruebas; dos causas (el webhook y la consulta cada minuto), como máximo 17 s después del push; duró 53 s

Dos integraciones se hicieron con pull request (la limpieza con Dask y la recurrencia, y el README del webhook); el resto se subió directo a `main`.

Limitaciones conocidas

Estas son las limitaciones que conocemos; las que dicen «no se probó» son vacíos de verificación, no fallos observados.

- La limpieza usa Dask sobre cada lote en un solo proceso; los workers del cluster de Dask solo intervienen en la comparativa.
- La ingesta (descargar, cargar y limpiar) se ejecuta a mano; Jenkins no la ejecuta.
- Las pruebas de la API dependen de los datos: las de `/spark/*` requieren que los dos jobs de Spark ya se hayan ejecutado.
- En la mayoría de los despliegues Jenkins recreó los contenedores de Dask, y alguna vez los de Spark, sin que cambiaran. No se determinó la causa, y por eso no se debe ejecutar un job mientras se hace un push.
- El relé se detiene si se reinicia el dev container; en ese caso el respaldo (consulta cada minuto) detecta el push, con una demora de hasta unos 72 s (build #22).
- El fallo provocado no cubre el caso de código roto en la API: la etapa Test recrea la API antes de correr las pruebas, y no se comprobó si esa API queda en marcha.
- `levantar.sh` solo se probó sobre un sistema ya en marcha, no desde cero.
- La comparativa lee los datos de forma distinta en cada motor, tiene 3 corridas por configuración y una sola medición de memoria por configuración.
- El polígono de Colombia está simplificado y no incluye las islas de San Andrés y Providencia.
- El cliente (Python 3.12.14) y el cluster de Dask (3.12.15) difieren en el parche; Dask lo avisa y no se observó ningún efecto.